

HarmonyOS ArkTS API 封装指导建议

Flask REST API 客户端封装方案

教学过程管理系统

目标

将已测试通过的 Flask 接口封装为简洁、优雅、可复用的 ArkTS API 层，使前端页面只调用业务方法，不接触 URL、Token、HTTP 状态码和 JSON 解析。

版本：V1.0

日期：2026 年 7 月

目录

1 封装目标与总体原则	3
1.1 关键设计原则	3
2 推荐目录结构	4
3 核心公共层	5
3.1 ApiConfig: 统一服务器配置	5
3.2 ApiTypes: 统一响应与通用类型	5
3.3 ApiError: 统一异常模型	6
3.4 QueryBuilder: 统一构造查询参数	7
3.5 SessionStore: Token 与会话持久化	7
4 统一网络层 ApiClient	9
4.1 ApiClient 不应承担的职责	13
5 Model 类型设计规范	13
5.1 字段命名约定	14
6 业务模块划分	14
7 业务模块示例	15
7.1 AuthApi	15
7.2 CourseApi	16
7.3 BookingApi	17
8 统一 Api 入口	18
8.1 应用初始化	19
9 调用示例	19
9.1 登录	19
9.2 查询课程并选课	19
9.3 查询空闲教室并提交预约	20
9.4 提交请假	20
9.5 审批预约	20
10 必须坚持的封装规则	21
11 交付内容与验收标准	21

11.1 交付给前端队友的内容	21
11.2 最终验收清单	21

1 封装目标与总体原则

最终结构应为：一个统一 **ApiClient** + 一个 **SessionStore** + 一套请求/响应类型 + 若干业务模块 + 一个统一 **Api** 入口。

页面或业务调用方只应看到如下代码：

```
await Api.auth.login('T001', '123456')
const courses = await Api.course.getCourses({ page: 1, pageSize: 20 })
await Api.booking.create({
  classroomId: 5,
  bookingDate: '2026-09-10',
  startSection: 3,
  endSection: 4,
  purpose: 'Class seminar',
  participantCount: 30
})
```

调用方不应直接处理以下内容：

- Flask 服务器地址和具体 URL；
- Bearer Token 的读取和请求头拼接；
- `http.createHttp()`、JSON 序列化与解析；
- HTTP 401、403、409 等状态码；
- Flask 统一响应中的 `code/message/data`；
- 网络错误和业务错误的类型转换。

1.1 关键设计原则

1. **单一网络入口**：所有接口必须经过同一个 **ApiClient**。
2. **模块化**：按认证、课程、预约、请假、调课、通知、管理等业务拆分。
3. **强类型**：请求参数和响应数据均定义 ArkTS 接口类型。
4. **自动鉴权**：除登录等公开接口外，自动携带 Token。
5. **统一错误**：所有异常统一转换为 **ApiError**。
6. **无页面依赖**：API 层不得调用路由、Toast 或页面组件。
7. **服务端为准**：客户端不自行决定最终权限和业务合法性。

2 推荐目录结构

在 entry/src/main/ets/api/ 下建立：

```
api/
├── core/
│   ├── ApiConfig.ets
│   ├── ApiTypes.ets
│   ├── ApiError.ets
│   ├── QueryBuilder.ets
│   ├── SessionStore.ets
│   └── ApiClient.ets
├── model/
│   ├── CommonModel.ets
│   ├── AuthModel.ets
│   ├── ProfileModel.ets
│   ├── NotificationModel.ets
│   ├── CourseModel.ets
│   ├── BookingModel.ets
│   ├── LeaveModel.ets
│   ├── ScheduleChangeModel.ets
│   └── AdminModel.ets
├── module/
│   ├── AuthApi.ets
│   ├── ProfileApi.ets
│   ├── CommonApi.ets
│   ├── NotificationApi.ets
│   ├── CourseApi.ets
│   ├── BookingApi.ets
│   ├── LeaveApi.ets
│   ├── ScheduleChangeApi.ets
│   └── AdminApi.ets
└── Api.ets
```

目录	职责
core	公共基础设施：服务器配置、统一类型、异常、查询参数、会话存储和网络请求。
model	每个业务模块的请求类型、响应类型、分页类型和枚举。
module	具体业务接口，例如登录、选课、提交预约、审批请假。

目录	职责
Api.ets	创建统一实例，对外集中暴露所有模块。

3 核心公共层

3.1 ApiConfig：统一服务器配置

```
export class ApiConfig {
  static readonly BASE_URL: string =
    'http://192.168.1.100:8080/api'

  static readonly CONNECT_TIMEOUT: number = 10000
  static readonly READ_TIMEOUT: number = 15000
  static readonly DEBUG: boolean = true
}
```

注意：真机访问电脑上的 Flask 服务时，不能使用 127.0.0.1 或 localhost，必须使用电脑的局域网 IP。建议 BASE_URL 已包含 /api，业务模块只写 /auth/login、/courses 等相对路径。

3.2 ApiTypes：统一响应与通用类型

```
export interface ApiResponse<T> {
  code: number
  message: string
  data: T | null
}

export interface PageResult<T> {
  list: Array<T>
  page: number
  pageSize: number
  total: number
}

export type QueryValue = string | number | boolean

export interface QueryParams {
  [key: string]: QueryValue | null | undefined
}

export interface EmptyData {
```

```
}

export enum UserRole {
  STUDENT = 'STUDENT',
  TEACHER = 'TEACHER',
  ACADEMIC_STAFF = 'ACADEMIC_STAFF',
  ADMIN = 'ADMIN'
}
```

分页字段必须和 Flask 实际返回保持完全一致。如果服务端返回 `items`，则不要在客户端定义为 `list`。

3.3 ApiError：统一异常模型

```
export class ApiError extends Error {
  readonly code: number
  readonly httpStatus: number
  readonly details: Object | null

  constructor(
    code: number,
    message: string,
    httpStatus: number = 0,
    details: Object | null = null
  ) {
    super(message)
    this.name = 'ApiError'
    this.code = code
    this.httpStatus = httpStatus
    this.details = details
  }

  isUnauthorized(): boolean {
    return this.httpStatus === 401 ||
      this.code === 40101 ||
      this.code === 40102
  }

  isForbidden(): boolean {
    return this.httpStatus === 403
  }

  isConflict(): boolean {
    return this.httpStatus === 409
  }
}
```

调用方只需要捕获 `ApiError`，无需区分底层网络错误、解析错误和 `Flask` 业务错误。

3.4 QueryBuilder：统一构造查询参数

```
import { QueryParams, QueryValue } from './ApiTypes'

export class QueryBuilder {
  static build(query?: QueryParams): string {
    if (query === undefined) {
      return ''
    }

    const parts: Array<string> = []

    Object.keys(query).forEach((key: string) => {
      const value: QueryValue | null | undefined = query[key]
      if (value !== undefined && value !== null) {
        parts.push(
          `${encodeURIComponent(key)}=` +
          `${encodeURIComponent(String(value))}`
        )
      }
    })

    return parts.length === 0 ? '' : `?${parts.join('&')}`
  }
}
```

业务模块不得手动拼接 `?`、`&` 和 URL 编码。

3.5 SessionStore：Token 与会话持久化

API 层使用 `Preferences` 保存少量会话信息：

- Token;
- 用户 ID;
- 账号、姓名、角色;
- Token 过期时间。

```
import { common } from '@kit.AbilityKit'
import { preferences } from '@kit.ArkData'
import { UserRole } from './ApiTypes'
```



```
export interface SessionInfo {
  token: string
  userId: number
  account: string
  realName: string
  role: UserRole
  expiresAt: string
}

export class SessionStore {
  private static readonly STORE_NAME: string =
    'teaching_api_session'

  private static store: preferences.Preferences | null = null

  static async init(context: common.Context): Promise<void> {
    SessionStore.store = await preferences.getPreferences(
      context,
      { name: SessionStore.STORE_NAME }
    )
  }

  private static requireStore(): preferences.Preferences {
    if (SessionStore.store === null) {
      throw new Error('SessionStore is not initialized')
    }
    return SessionStore.store
  }

  static async save(session: SessionInfo): Promise<void> {
    const store = SessionStore.requireStore()
    await store.put('token', session.token)
    await store.put('userId', session.userId)
    await store.put('account', session.account)
    await store.put('realName', session.realName)
    await store.put('role', session.role)
    await store.put('expiresAt', session.expiresAt)
    await store.flush()
  }

  static async getToken(): Promise<string> {
    const value = await SessionStore.requireStore().get('token', '')
    return String(value)
  }

  static async clear(): Promise<void> {
```

```

const store = SessionStore.requireStore()
await store.clear()
await store.flush()
}
}

```

API 层只负责 Preferences 持久化，不直接依赖 AppStorage。AppStorage 属于页面运行状态，不应成为网络层依赖。

4 统一网络层 ApiClient

所有 Flask 接口最终必须经过同一个 ApiClient。它只负责：

- 拼接 URL；
- 自动添加 Token；
- 序列化请求体；
- 解析统一响应；
- 转换为统一异常；
- 处理登录失效；
- 销毁 HTTP 请求对象。

```

import { http } from '@kit.NetworkKit'
import { BusinessError } from '@kit.BasicServicesKit'
import { ApiConfig } from './ApiConfig'
import { ApiError } from './ApiError'
import { ApiResponse, QueryParams } from './ApiTypes'
import { QueryBuilder } from './QueryBuilder'
import { SessionStore } from './SessionStore'

export interface RequestOptions {
  auth?: boolean
  query?: QueryParams
  body?: Object
}

export type UnauthorizedHandler = () => void

export class ApiClient {
  private unauthorizedHandler: UnauthorizedHandler | null = null

  setUnauthorizedHandler(handler: UnauthorizedHandler): void {

```

```
    this.unauthorizedHandler = handler
  }

  async get<T>(
    path: string,
    query?: QueryParams,
    auth: boolean = true
  ): Promise<T> {
    return this.request<T>(path, http.RequestMethod.GET, {
      auth: auth,
      query: query
    })
  }

  async post<T>(
    path: string,
    body?: Object,
    auth: boolean = true
  ): Promise<T> {
    return this.request<T>(path, http.RequestMethod.POST, {
      auth: auth,
      body: body
    })
  }

  async put<T>(
    path: string,
    body?: Object,
    auth: boolean = true
  ): Promise<T> {
    return this.request<T>(path, http.RequestMethod.PUT, {
      auth: auth,
      body: body
    })
  }

  async delete<T>(
    path: string,
    body?: Object,
    auth: boolean = true
  ): Promise<T> {
    return this.request<T>(path, http.RequestMethod.DELETE, {
      auth: auth,
      body: body
    })
  }
}
```

```
private async request<T>(  
  path: string,  
  method: http.RequestMethod,  
  options: RequestOptions  
) : Promise<T> {  
  const request = http.createHttp()  
  
  try {  
    const headers: Record<string, string> = {  
      'Accept': 'application/json',  
      'Content-Type': 'application/json'  
    }  
  
    if (options.auth !== false) {  
      const token = await SessionStore.getToken()  
      if (token.length === 0) {  
        throw new ApiError(40101, 'Not logged in', 401)  
      }  
      headers['Authorization'] = `Bearer ${token}`  
    }  
  
    const url = `${ApiConfig.BASE_URL}${path}` +  
      QueryBuilder.build(options.query)  
  
    const response = await request.request(url, {  
      method: method,  
      header: headers,  
      extraData: options.body === undefined  
        ? undefined  
        : JSON.stringify(options.body),  
      expectDataType: http.HttpDataType.STRING,  
      connectTimeout: ApiConfig.CONNECT_TIMEOUT,  
      readTimeout: ApiConfig.READ_TIMEOUT  
    })  
  
    const statusCode = response.responseCode  
    if (typeof response.result !== 'string') {  
      throw new ApiError(50001, 'Invalid response', statusCode)  
    }  
  
    let envelope: ApiResponse<T>  
    try {  
      envelope = JSON.parse(response.result) as ApiResponse<T>  
    } catch (_error) {  
      throw new ApiError(50001, 'JSON parse failed', statusCode)  
    }  
  }  
}
```

```
}

if (statusCode === 401 ||
    envelope.code === 40101 ||
    envelope.code === 40102) {
    await SessionStore.clear()
    if (this.unauthorizedHandler !== null) {
        this.unauthorizedHandler()
    }
    throw new ApiError(
        envelope.code,
        envelope.message || 'Session expired',
        statusCode,
        envelope.data as Object | null
    )
}

if (statusCode < 200 || statusCode >= 300) {
    throw new ApiError(
        envelope.code,
        envelope.message || `HTTP ${statusCode}`,
        statusCode,
        envelope.data as Object | null
    )
}

if (envelope.code !== 0) {
    throw new ApiError(
        envelope.code,
        envelope.message,
        statusCode,
        envelope.data as Object | null
    )
}

return envelope.data as T
} catch (error) {
    if (error instanceof ApiError) {
        throw error
    }
    const businessError = error as BusinessError
    throw new ApiError(
        50000,
        businessError.message || 'Network request failed'
    )
} finally {
```

```
        request.destroy()
    }
}
}
```

4.1 ApiClient 不应承担的职责

- 不调用页面路由；
- 不显示 Toast；
- 不管理加载动画；
- 不缓存课程、通知、审批列表；
- 不根据本地角色决定最终权限；
- 不记录密码、Authorization 请求头或完整 Token。

5 Model 类型设计规范

不要把请求字段、列表字段和详情字段混在一个大接口中。应拆分为：

- 查询条件：CourseQuery；
- 列表项：CourseSummary；
- 详情：CourseDetail；
- 新建请求：CreateCourseRequest；
- 更新请求：UpdateCourseRequest。

```
import { PageResult, QueryParams } from '../core/ApiTypes'

export interface CourseQuery extends QueryParams {
    page?: number
    pageSize?: number
    selectionStatus?: string
    departmentId?: number
    keyword?: string
}

export interface CourseSchedule {
    id: number
    semester: string
    classroomId: number
    classroomName: string
    startWeek: number
}
```

```
endWeek: number
weekDay: number
startSection: number
endSection: number
weekType: string
}

export interface CourseSummary {
  id: number
  courseCode: string
  courseName: string
  teacherId: number
  teacherName: string
  credit: number
  capacity: number
  selectedCount: number
  remainingCount: number
  selectionStatus: string
  schedules: Array<CourseSchedule>
}

export interface CourseDetail extends CourseSummary {
  departmentId: number
  departmentName: string
  description: string | null
}

export type CoursePage = PageResult<CourseSummary>
```

5.1 字段命名约定

层级	示例	风格	说明
数据库	real_name	snake_case	SQLite 字段
Python 内部	real_name	snake_case	Flask 变量
JSON 接口	realName	camelCase	前后端传输
ArkTS	realName	camelCase	Model 字段

6 业务模块划分

模块	主要职责
AuthApi	登录、当前用户、退出、修改密码。所有账号由管理员预先创建，不提供注册接口。
ProfileApi	查询和更新个人资料，获取学生、教师、教务主页聚合数据。
CommonApi	院系、班级、学期、教室、空闲教室、个人课表等通用查询。
NotificationApi	接收通知、未读数、详情、已读、发布、已发布列表、管理列表、撤回。
CourseApi	课程查询、详情、排课、学生选课/退课、教师授课课程和学生名单。
BookingApi	教室预约提交、我的预约、详情、取消、日志、待审批、通过、驳回。
LeaveApi	学生请假、我的请假、详情、取消、教师相关请假、教务审批。
ScheduleChangeApi	教师调课申请、记录、详情、冲突、取消、教务审批。
AdminApi	人员、院系、班级、教室、学期、课程、排课和审批历史管理。

7 业务模块示例

7.1 AuthApi

```
import { ApiClient } from '../core/ApiClient'
import { EmptyData } from '../core/ApiTypes'
import { SessionInfo, SessionStore } from '../core/SessionStore'
import { ChangePasswordRequest, CurrentUser, LoginResult }
  from '../model/AuthModel'

export class AuthApi {
  constructor(private readonly client: ApiClient) {
  }

  async login(account: string, password: string): Promise<LoginResult> {
    const result = await this.client.post<LoginResult>(
      '/auth/login',
      { account: account, password: password },
      false
    )

    const session: SessionInfo = {
      token: result.token,
      userId: result.userId,
```



```
        account: result.account,
        realName: result.realName,
        role: result.role,
        expiresAt: result.expiresAt
    }

    await SessionStore.save(session)
    return result
}

async me(): Promise<CurrentUser> {
    return this.client.get<CurrentUser>('/auth/me')
}

async logout(): Promise<void> {
    try {
        await this.client.post<EmptyData>('/auth/logout')
    } finally {
        await SessionStore.clear()
    }
}

async changePassword(request: ChangePasswordRequest): Promise<void> {
    await this.client.put<EmptyData>('/auth/password', request)
}
```

7.2 CourseApi

```
export class CourseApi {
    constructor(private readonly client: ApiClient) {}

    async getCourses(query: CourseQuery = {}): Promise<CoursePage> {
        return this.client.get<CoursePage>('/courses', query)
    }

    async getCourse(courseId: number): Promise<CourseDetail> {
        return this.client.get<CourseDetail>(`/courses/${courseId}`)
    }

    async enroll(courseId: number): Promise<void> {
        await this.client.post<EmptyData>(`/courses/${courseId}/enroll`)
    }
}
```

```
async drop(courseId: number): Promise<void> {
  await this.client.post<EmptyData>(`/courses/${courseId}/drop`)
}

async getTeachingCourses(): Promise<Array<CourseSummary>> {
  return this.client.get<Array<CourseSummary>>(`/teaching/courses`)
}

async getTeachingStudents(
  courseId: number
): Promise<Array<CourseStudent>> {
  return this.client.get<Array<CourseStudent>>(
    `/teaching/courses/${courseId}/students`
  )
}
}
```

7.3 BookingApi

```
export class BookingApi {
  constructor(private readonly client: ApiClient) {
  }

  async create(request: CreateBookingRequest): Promise<BookingDetail> {
    return this.client.post<BookingDetail>(`/bookings`, request)
  }

  async getMyBookings(
    query: BookingQuery = {}
  ): Promise<BookingPage> {
    return this.client.get<BookingPage>(`/bookings/my`, query)
  }

  async cancel(bookingId: number, reason: string): Promise<void> {
    await this.client.post<EmptyData>(
      `/bookings/${bookingId}/cancel`,
      { reason: reason }
    )
  }

  async approve(bookingId: number, comment?: string): Promise<void> {
    await this.client.post<EmptyData>(
      `/admin/bookings/${bookingId}/approve`,
      { comment: comment }
    )
  }
}
```

```

}

async reject(bookingId: number, comment: string): Promise<void> {
  await this.client.post<EmptyData>(
    `/admin/bookings/${bookingId}/reject`,
    { comment: comment }
  )
}
}

```

8 统一 Api 入口

整个项目只创建一个 ApiClient 和一个 Api 实例。

```

import { common } from '@kit.AbilityKit'
import { ApiClient } from './core/ApiClient'
import { SessionStore } from './core/SessionStore'
import { AuthApi } from './module/AuthApi'
import { ProfileApi } from './module/ProfileApi'
import { CommonApi } from './module/CommonApi'
import { NotificationApi } from './module/NotificationApi'
import { CourseApi } from './module/CourseApi'
import { BookingApi } from './module/BookingApi'
import { LeaveApi } from './module/LeaveApi'
import { ScheduleChangeApi } from './module/ScheduleChangeApi'
import { AdminApi } from './module/AdminApi'

class TeachingApi {
  private readonly client: ApiClient

  readonly auth: AuthApi
  readonly profile: ProfileApi
  readonly common: CommonApi
  readonly notification: NotificationApi
  readonly course: CourseApi
  readonly booking: BookingApi
  readonly leave: LeaveApi
  readonly scheduleChange: ScheduleChangeApi
  readonly admin: AdminApi

  constructor() {
    this.client = new ApiClient()
    this.auth = new AuthApi(this.client)
    this.profile = new ProfileApi(this.client)
    this.common = new CommonApi(this.client)
  }
}

```

```
this.notification = new NotificationApi(this.client)
this.course = new CourseApi(this.client)
this.booking = new BookingApi(this.client)
this.leave = new LeaveApi(this.client)
this.scheduleChange = new ScheduleChangeApi(this.client)
this.admin = new AdminApi(this.client)
}

async init(context: common.Context): Promise<void> {
  await SessionStore.init(context)
}

onUnauthorized(handler: () => void): void {
  this.client.setUnauthorizedHandler(handler)
}
}

export const Api = new TeachingApi()
```

8.1 应用初始化

在 EntryAbility 中只初始化一次：

```
await Api.init(this.context)

Api.onUnauthorized(() => {
  console.info('Session expired')
})
```

未登录回调只负责发出信号。API 层不得直接执行页面跳转。

9 调用示例

9.1 登录

```
const session = await Api.auth.login('20240001', '123456')
```

9.2 查询课程并选课

```
const page = await Api.course.getCourses({
  page: 1,
  pageSize: 20,
  selectionStatus: 'OPEN'
})
```

```
await Api.course.enroll(page.list[0].id)
```

9.3 查询空闲教室并提交预约

```
const classrooms = await Api.common.getAvailableClassrooms({
  date: '2026-09-10',
  startSection: 3,
  endSection: 4,
  participantCount: 30
})

await Api.booking.create({
  classroomId: classrooms[0].id,
  bookingDate: '2026-09-10',
  startSection: 3,
  endSection: 4,
  purpose: 'Class seminar',
  participantCount: 30
})
```

9.4 提交请假

```
await Api.leave.create({
  leaveType: 'SICK',
  startDate: '2026-09-10',
  endDate: '2026-09-10',
  startSection: 1,
  endSection: 4,
  reason: 'Not feeling well',
  courseIds: [2, 5]
})
```

9.5 审批预约

```
await Api.booking.reject(
  bookingId,
  'The classroom is reserved for a meeting'
)
```

10 必须坚持的封装规则

1. 调用方不得出现 URL。只能调用 `Api.course.getCourses()` 等业务方法。
2. 调用方不得手动传 Token。Token 由 `ApiClient` 自动读取。
3. 调用方不得解析统一响应。业务模块直接返回 data。
4. 无返回数据的方法使用 `Promise<void>`。
5. API 层不得显示 Toast 或跳转页面。
6. 最终权限由 Flask 判断。客户端本地角色只能辅助界面展示。
7. 请求和响应类型必须分开。
8. JSON 与 ArkTS 字段统一使用 camelCase。
9. 路径只出现在对应业务模块中。
10. 禁止记录密码、完整 Token 和 Authorization 请求头。

11 交付内容与验收标准

11.1 交付给前端队友的内容

- 完整的 api/ 目录；
- API 初始化说明；
- Flask 地址配置位置；
- 每个模块的公开方法清单；
- 请求与响应 Model 类型；
- 常用调用示例；
- 错误处理说明；
- Token 失效回调说明。

11.2 最终验收清单

序号	验收项
1	前端只需导入 <code>Api.ets</code> 即可调用所有模块。
2	所有网络请求经过同一个 <code>ApiClient</code> 。
3	所有鉴权接口自动携带 Bearer Token。

序号	验收项
4	HTTP 401 或 Token 业务错误自动清除本地会话。
5	所有错误统一转换为 <code>ApiError</code> 。
6	所有请求和响应均有明确 ArkTS 类型。
7	分页接口统一使用同一个分页结构。
8	API 层不引用页面组件、Toast 或 Router。
9	API 层不包含页面状态和业务列表缓存。
10	已测试通过的全部 Flask 接口均能在对应业务模块中找到。
11	所有账号由管理员预先创建， <code>AuthApi</code> 不提供注册方法。
12	业务方法名称清晰、短小，每个方法只负责一次 HTTP 调用。

最终效果：前端队友不需要阅读几十个接口的 HTTP 细节，只需查看模块方法签名和 Model 类型，即可安全、稳定地完成页面开发。